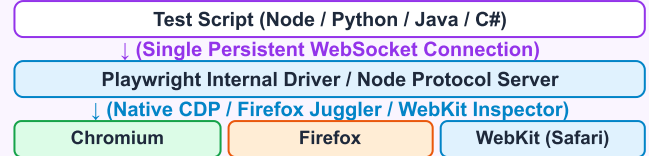


1. Introduction to Playwright & Architecture

1 What is Playwright?

- Modern open-source E2E framework engineered by **Microsoft** (2020).
- Cross-browser testing across **Chromium, Firefox & WebKit** with one API.
- Multi-language support: **TypeScript, JavaScript, Python, Java, C# (.NET)**.
- Connects over bi-directional **WebSocket (CDP/Juggler)** without HTTP drivers.
- Engineered ground-up for dynamic SPAs: React, Angular, Vue, Next.js, Remix.

2 Playwright Internal Architecture



3 Selenium vs Playwright vs Cypress

Feature	Playwright	Selenium	Cypress
Protocol	Bi-dir WebSocket	HTTP JSON wire	In-browser JS
Auto-Wait	Native 7-point checks	Manual explicit waits	DOM-only retry
Multi-Tabs	Native 1st class	Window handle loops	No multi-tab support
Multi-Domains	Full multi-origin	Supported	Restricted origin
Execution	Ultra-fast parallel	Heavy process hops	Single thread queue

4 Core Building Blocks Hierarchy

- Browser:** Heavy OS process instance (e.g. Chromium). Spawned once per worker.
- BrowserContext:** Ultra-fast, isolated incognito sandbox (~10ms). Never shares cookies.
- Page:** Single browser tab inside context.

```
const browser = await chromium.launch({ headless: true }); const context = await browser.newContext(); // Isolated Sandbox const page = await context.newPage(); // Tab inside context await page.goto('https://app.dev');
```

5 High-Yield SDET Interview Flashcards

Q1: Why does Playwright NOT need ChromeDriver or GeckoDriver?

Ans: Playwright bundles patched native browser binaries and establishes a direct socket connection over debugging protocols, eliminating driver translation overhead.

Q2: How does Playwright guarantee zero flakiness without sleeps?

Ans: Through **Actionability Checks** (waiting for visibility, stability, enablement, receiver check) and **Web-First retrying assertions**.

Q3: What makes BrowserContext so powerful for parallel CI test execution?

Ans: Contexts provide total separation of cache, local storage, sessions, and cookies without paying the resource cost of opening separate browser executables.

6 Key SDET Architectural Takeaways

Zero Driver Headaches

Bundled binaries eliminate mismatched browser/driver version crashes.

WebSocket Bidirectionality

Browser pushes events asynchronously without round-trip polling delay.

Built for Scale

Parallel execution per core natively maximizes CI compute ROI.

"Direct WebSocket protocol + BrowserContext isolation = Unbeatable E2E speed and reliability!"

2. Browser Setup & Environment Config

1 Installation & Master CLI Commands

- **Init Scaffold:** `npm init playwright@latest`
- **Install Browsers:** `npx playwright install --with-deps`
- **Install Specific:** `npx playwright install webkit`
- **Run All Specs:** `npx playwright test`
- **Run Headed:** `npx playwright test --headed`
- **Run Specific File:** `npx playwright test tests/login.spec.ts`
- **Run Specific Browser:** `npx playwright test --project=firefox`
- **UI Interactive Mode:** `npx playwright test --ui`

2 Browser Launch Arguments & Channels

```
import { chromium } from '@playwright/test';
const browser = await chromium.launch({ headless:
false, // Visible browser UI for local dev slowMo:
50, // Slow down by 50ms per step devtools: false,
// Auto open Chrome DevTools panel args: ['--start-
maximized', '--no-sandbox'], channel: 'chrome', //
Target installed Google Chrome (or 'msedge') proxy:
{ server: 'http://proxy.corp.com:8080' } });
```

Channel Tip: Use channel: 'msedge' to test Enterprise Edge SSO!

3 playwright.config.ts Production Master Anatomy

```
import { defineConfig, devices } from '@playwright/test'; export default defineConfig({ testDir:
'./tests', timeout: 30 * 1000, expect: { timeout: 5000 }, fullyParallel: true, // Parallelism within single
test files forbidOnly: !!process.env.CI, // Prevent accidental test.only merges to main retries: process.env.CI
? 2 : 0, // Auto-retries on CI build agents workers: process.env.CI ? 2 : undefined, // Thread allocation for
CPU scaling reporter: [['html', { open: 'never' }], ['list'], ['junit', { outputFile: 'results.xml' }]], use: {
baseUrl: process.env.STAGING ? 'https://staging.app.com' : 'https://prod.app.com', trace: 'on-first-retry', //
Low overhead trace recording screenshot: 'only-on-failure', video: 'retain-on-failure', viewport: { width:
1280, height: 720 }, ignoreHTTPSErrors: true, }, projects: [ { name: 'chromium', use: { ...devices['Desktop
Chrome'] } }, { name: 'firefox', use: { ...devices['Desktop Firefox'] } }, { name: 'webkit', use: {
...devices['Desktop Safari'] } }, { name: 'mobile', use: { ...devices['Pixel 5'] } }, ], });
```

4 Context Device Emulation Features

- **Geolocation & Perms:** `geolocation: { lat: 37.7, long: -122.4 }, permissions: ['geolocation']`
- **Timezone & Locale:** `locale: 'fr-FR', timezoneId: 'Europe/Paris'`
- **Color Scheme:** `colorScheme: 'dark' (Test Dark Theme)`
- **Offline:** `offline: true (PWA service worker caching)`

5 Interview Gotcha: Browser vs Context

Q: Can two parallel tests share the same BrowserContext?
Never! Sharing a context creates state pollution (cookies, local storage, cart items bleed into other tests). Always create an independent context per test worker via test fixtures!

6 Multi-Environment Secrets & Dotenv Setup

In enterprise suites, load environmental config files dynamically before test suite boot:

```
import dotenv from 'dotenv'; import path from 'path'; dotenv.config({ path: path.resolve(__dirname,
`.env.${process.env.ENV || 'staging'}`) }); // Usage: baseUrl: process.env.BASE_URL, storageState:
process.env.AUTH_FILE
```

"Always run headless in CI and use on-first-retry tracing for zero-overhead triage!"

3. Playwright Locators & ARIA Philosophy

1 What is a Locator? (Lazy Evaluation & Strict Mode)

- A Locator is a **live pointer** to an element on the page at any given moment.
- Lazy Evaluation:** Locators do NOT query the DOM when defined. The DOM is queried only when an action (e.g. `.click()`) is invoked!
- Strict Mode:** If a locator resolves to more than 1 matching DOM node, Playwright immediately throws a **Strictness Violation error** to prevent indeterminate clicks.

2 Official User-Facing Recommended Locators

Locator Method	DOM Target Match	Best Practice Example
<code>getByRole()</code> [#1 Best]	ARIA accessibility role & name	<code>page.getByRole('button', { name: 'Submit Application' })</code>
<code>getByText()</code>	Visible text content	<code>page.getByText('Order Confirmation', { exact: true })</code>
<code>getByLabel()</code>	Associated <code><label></code> input text	<code>page.getByLabel('Work Email Address')</code>
<code>getByPlaceholder()</code>	Input placeholder attribute	<code>page.getByPlaceholder('Search products...')</code>
<code>getByTestId()</code>	<code>data-testid</code> attribute	<code>page.getByTestId('shopping-cart-badge')</code>
<code>getByAltText()</code> / <code>getTitle()</code>	Image alt text / element tooltip	<code>page.getByAltText('Company Profile Banner')</code>

3 Filtering & Chaining Locators

```
// Chain parent to child:
page.getByRole('listitem').filter({ hasText:
  'Product 12' }).getByRole('button', { name: 'Add to
  Cart' }).click(); // Filter by child element
presence: page.getByRole('row').filter({ has:
  page.getByRole('heading', { name: 'Admin' }) }); //
  Filter by not having element:
page.getByRole('listitem').filter({ hasNot:
  page.getByText('Out of Stock' ) });
```

4 Resolving Strict Mode Violations

- `.first()` : Pick first matched element
- `.last()` : Pick last matched element
- `.nth(index)` : Pick by 0-based index
- `.count()` : Returns total matched count

```
const row = page.getByRole('row').nth(2);
const total = await page.getByRole('row').count();
// Pick exact among duplicates:
page.getByRole('button', { name: 'Save', exact: true
  });
```

5 Interview Deep Dive: Why XPath is Deprecated in Modern Automation

Brittle DOM Hierarchy: `/div[2]/div/form/div[3]/button` breaks the moment a developer wraps an element inside an extra layout container or changes a CSS class.

Accessible Locators Validate UX: `getByRole('button', { name: 'Pay' })` tests the DOM as a screen reader and user perceives it. If the button is not accessible, the test fails!

"Locate by user role and accessible name, never by brittle DOM hierarchy!"

4. Finding Web Elements & Advanced Selectors

1 CSS & XPath Engine Support

- **CSS Selector:** `page.locator('button.btn-primary')`
- **Attribute Match:**
`page.locator('input[name="user_email"]')`
- **XPath Expression:**
`page.locator('//button[text()='Submit']')`
- **XPath Following-Sibling:**
`page.locator('//label[text()='Age']/following-sibling::input')`

```
// Substring attribute selectors:  
page.locator('a[href^="https://"]') // starts with  
page.locator('a[href$=".pdf"]') // ends with  
page.locator('a[href*="invoice"]') // contains
```

2 Playwright Pseudo-CSS Engines

- **:has-text():** Container containing inner string
- **:visible:** Matches only elements currently rendered visible
- **:right-of() / :left-of():** Relative visual layout anchors

```
// Exclusive Playwright pseudo-selectors:  
page.locator('button:has-text("Sign In")');  
page.locator('.card:visible');  
page.locator('input:right-of(:text("Username"))');  
page.locator('button:below(:text("Accept Terms"))');
```

3 Shadow DOM & Pierce-Through Engine

Automatic Piercing: Unlike Selenium which requires `driver.getShadowRoot()`, Playwright CSS selectors seamlessly pierce open Shadow DOM boundaries automatically!

```
// Pierces shadow roots automatically: await  
page.locator('custom-calendar .day-active').click();  
await page.locator('shadow-host  
input#token').fill('123');
```

4 Iterating & Extracting Lists

```
const items = page.getByRole('listitem');  
const count = await items.count(); // Iterating over  
dynamic items for (let i = 0; i < count; ++i) {  
  console.log(await items.nth(i).textContent()); } //  
Bulk extraction in 1 CDP roundtrip: const allTexts =  
await items.allTextContents(); const allInners =  
await items.allInnerTexts();
```

5 page.\$\$ vs locator.all()

Approach	Type Returned	Auto-waiting
<code>page.\$\$(...)</code>	<code>ElementHandle[]</code>	No (Discouraged)
<code>locator.all(...)</code>	<code>Locator[]</code>	Yes (Recommended)

```
for (const card of await  
page.getByTestId('card').all()) { await  
card.click(); // Has auto-waiting! }
```

6 Custom TestId Attribute Registration

If your company uses `data-cy` or `qa-id` instead of `data-testid`, configure it globally in setup:

```
import { selectors } from '@playwright/test'; selectors.setTestIdAttribute('data-qa'); // Now  
page.getByTestId('login') matches data-qa="login"!
```

"Automatic Shadow DOM piercing eliminates 90% of legacy automation headaches!"

5. Core Element Methods & Actions

1 Actionability Checks (The Under-The-Hood Engine)

Before executing ANY action (e.g. click), Playwright automatically validates these **Actionability Criteria**:

- | | | | |
|---------------------------|---------------------------|-----------------------------------|-----------------------------|
| 1. Attached to DOM | 2. Visible (Non-zero box) | 3. Stable (Not animating) | 4. Receives Pointer Events |
| 5. Enabled (Not disabled) | 6. Editable (For input) | 7. Within Viewport (Auto-scrolls) | 8. Action Timeout Respected |

2 Input & Click Actions

```
// Fill: clears existing + pastes value
instantly await
page.getByLabel('Username').fill('john_doe'); //
Type / PressSequentially (mimics slow human typing)
await
page.getByLabel('Search').pressSequentially('iphone'
, { delay: 100 }); // Clicks: await
page.getByRole('button').click(); await
page.getByRole('button').dblclick(); await
page.getByRole('button').click({ button: 'right' });
await page.getByRole('button').click({ modifiers:
['Shift'] }); await page.getByRole('button').click({
position: { x: 5, y: 5 } });
```

3 State Toggles & Checkboxes

```
const checkbox = page.getByLabel('I agree to
Terms'); // Idempotent check (won't uncheck if
already checked!) await checkbox.check(); await
checkbox.uncheck(); await checkbox.setChecked(true);
// Radio selection: await page.getByLabel('Credit
Card').check(); // State assertions: await
expect(checkbox).toBeChecked(); await
expect(checkbox).toBeDisabled();
```

4 Reading Element Data & Attributes

```
// Read inner rendered text: const text =
await page.getByRole('heading').innerText(); // Read
input value (state): const val = await
page.getByLabel('Email').inputValue(); // Read
attribute: const href = await
page.getByRole('link').getAttribute('href'); // Read
CSS property: const color = await
page.locator('.btn').evaluate(el =>
window.getComputedStyle(el).backgroundColor );
```

5 Force Action Bypass ({ force: true })

When a non-interactive transparent overlay or third-party modal obscures an element:

```
await page.getByRole('button', { name:
'Submit' }) .click({ force: true });
```

Notice: force:true bypasses actionable checks. Never use as quick fix for race conditions!

6 Focus, Blur & Dispatching Custom DOM Events

```
// Focus and trigger blur event: await page.getByLabel('Tax ID').focus(); await page.getByLabel('Tax
ID').blur(); // Dispatch direct custom event (e.g., custom change listeners): await page.locator('#rating-
star').dispatchEvent('rating-updated', { stars: 5 });
```

"Actionability guarantees zero 'ElementClickIntercepted' exceptions!"

6. Dialogs, Alerts & JavaScript Popups

1 Default Auto-Dismissal Behavior

- Unlike Selenium where unhandled alerts throw `UnhandledAlertException` and freeze test execution, Playwright **automatically dismisses** all dialogs by default!
- To accept, inspect message content, or fill a prompt, register an event listener **BEFORE** triggering the action!

2 Handling Alert & Confirm Dialogs

```
// Handle Alert (Accept/OK):
page.once('dialog', async dialog => {
  console.log(`Alert text: ${dialog.message()}`);
  expect(dialog.message()).toBe('Changes saved!');
  await dialog.accept(); }); await
page.getByRole('button', { name: 'Save' }).click();
// Handle Confirm (Cancel/Dismiss):
page.once('dialog', async dialog => {
  expect(dialog.type()).toBe('confirm'); await
  dialog.dismiss(); // Clicks "Cancel" }); await
page.getByRole('button', { name: 'Delete Account'
}).click();
```

3 Handling Prompt Dialog with Input

```
// Handle Prompt with custom text:
page.once('dialog', async dialog => {
  expect(dialog.type()).toBe('prompt');
  expect(dialog.defaultValue()).toBe('Guest'); await
  dialog.accept('Playwright SDET'); // inputs text!
}); await page.getByRole('button', { name: 'Set
Nickname' }).click(); // Verify result on page:
await expect(page.getByText('Hello, Playwright
SDET')).toBeVisible();
```

4 Dialog Types Reference Table

Dialog Type	JS Invocation	Default Handler
alert	<code>window.alert()</code>	Auto-dismissed
confirm	<code>window.confirm()</code>	Auto-dismissed (Cancel)
prompt	<code>window.prompt()</code>	Auto-dismissed (Cancel)
beforeunload	Unsaved navigate	Auto-accepted

5 The Fatal Listener Timing Trap

Common Interview Bug: Attaching listener **AFTER** clicking the button!

```
// ❌ DEADLOCK / MISSED EVENT: await
page.getByRole('button').click(); // Dialog opens &
auto-dismisses! page.on('dialog', ...); // Never
called! // ✅ CORRECT: page.once('dialog', async d
=> await d.accept()); await
page.getByRole('button').click();
```

6 Handling Native Browser Permissions (Camera, Mic, Notifications)

Native OS permission dialogs are NOT JS alerts! Grant them directly on the browser context:

```
const context = await browser.newContext({ permissions: ['camera', 'microphone', 'notifications',
'clipboard-read' ] });
```

"Always register your `page.once('dialog')` BEFORE triggering the click!"

7. Frames & iFrames Handling

1 The FrameLocator Paradigm Shift

- In Selenium, you had to call `driver.switchTo().frame()` and remember to call `switchTo().defaultContent()` afterwards.
- Playwright introduces `page.frameLocator()`: A scoped locator that stays inside the frame, automatically waits for iframe load, and requires **zero context switching**!

2 Modern frameLocator Approach

```
// 1. Locate iframe by CSS selector, ID or
name: const stripeFrame =
page.frameLocator('#stripe-payment'); // 2. Interact
directly with elements inside: await
stripeFrame.getByLabel('Card Number').fill('4242
4242'); await stripeFrame.getByRole('button', {
name: 'Pay' }).click(); // 3. Seamlessly interact
with top page immediately: await
page.getByText('Payment Succeeded').waitFor();
```

3 Deeply Nested Frames

```
// Frame inside another frame (chaining):
const parentFrame =
page.frameLocator('#parentIframe'); const childFrame
= parentFrame.frameLocator('#childIframe'); //
Interact with element inside child frame: await
childFrame.getByRole('button', { name: 'Confirm'
}).click(); // No switchTo().parentFrame() required!
```

4 Legacy Frame Object by URL/Name

```
// Find frame by URL regex or exact name:
const frame = page.frame({ url: /.recaptcha./ });
if (frame) { await
frame.locator('#checkbox').click(); } // Access main
root frame: const root = page.mainFrame();
```

5 Frame Detached & Re-render Handling

Because `frameLocator` is a lazy locator, if the iframe re-renders or refreshes its `src` dynamically, Playwright re-identifies the new frame instance on next action automatically without stale errors!

6 Inspecting All Active Frames in Page

```
const allFrames = page.frames(); console.log(`Total active frames on page: ${allFrames.length}`); for
(const f of allFrames) { console.log(`Frame URL: ${f.url()} | Name: ${f.name()}`); }
```

"No `switchTo().defaultContent()` nightmares — `frameLocator` does it all seamlessly!"

8. Multi-Tabs & Multiple Windows

1 How Playwright Manages Multiple Tabs

- A browser tab or popup window is represented as a first-class **Page object** within the same BrowserContext.
- They share cookies, session tokens, and local storage automatically.
- You never loop through window GUIDs like in Selenium; you capture the new page via `waitForEvent('page')`.

2 Gold Standard: Handling New Tab / Target="_blank" Link

```
// Wait for popup event simultaneously with the user click that opens it: const [newPage] = await
Promise.all([ context.waitForEvent('page'), page.getByRole('link', { name: 'Help Documentation' }).click() ]);
// Wait for load and interact with new tab directly: await newPage.waitForLoadState('domcontentloaded');
console.log(`New tab title: ${await newPage.title()}`); await newPage.getByRole('searchbox').fill('OAuth
guide'); await newPage.close(); // Close child tab // Immediately continue on primary original page: await
page.bringToFront(); await page.getByRole('button', { name: 'Done' }).click();
```

3 Accessing All Open Tabs Array

```
// Retrieve array of all active pages in
context: const allPages = context.pages();
console.log(`Total tabs open: ${allPages.length}`);
// Interact with specific tab: const originalPage =
allPages[0]; const secondPage = allPages[1]; await
secondPage.bringToFront();
```

4 Opening New Tab Programmatically

```
// Manually launch independent blank tab:
const adminTab = await context.newPage(); await
adminTab.goto('https://app.dev/admin'); // Toggle
between tabs: await page.bringToFront(); await
adminTab.bringToFront();
```

5 Handling Popups Triggered by Script (window.open)

```
// If page opens popup asynchronously via JS timer: const popupPromise = page.waitForEvent('popup');
await page.getByRole('button', { name: 'Auth with Google' }).click(); const popup = await popupPromise; await
popup.waitForLoadState(); await popup.fill('input[type="email"]', 'sdet@gmail.com');
```

"Tabs are independent Page objects: zero handle loops, pure concurrency!"

9. Single & Multi-Select Dropdowns

1 Native <select> Elements

Use `selectOption()`. Handles option selection by **label**, **value**, or **index**:

```
const dropdown =
page.locator('select#country'); // Select by visible
label text: await dropdown.selectOption({ label:
'United States' }); // Select by value attribute:
await dropdown.selectOption({ value: 'US' }); //
Select by 0-based index: await
dropdown.selectOption({ index: 3 }); // Shortcut by
direct string (matches value or label): await
dropdown.selectOption('Canada');
```

2 Multiple Select (<select multiple>)

Pass an array to select multiple options in one atomic step:

```
const multiSelect =
page.locator('select#skills'); // Select multiple
values simultaneously: await
multiSelect.selectOption(['playwright',
'typescript', 'docker']); // Select by mixed
objects: await multiSelect.selectOption([ { label:
'QA Lead' }, { value: 'dev_ops' } ]); //
Clear/deselect all options: await
multiSelect.selectOption([]);
```

3 Custom Non-Standard Dropdowns (Div / UI / Li / React Select)

Modern frameworks (MUI, AntD, Tailwind) construct custom DOMs using pseudo-selects:

```
// Step 1: Click the trigger wrapper to expand dropdown overlay await page.getByRole('combobox', { name:
'Choose Department' }).click(); // Step 2: Click the target option from the floating portal list await
page.getByRole('option', { name: 'Engineering' }).click(); // Step 3: Handle search-filterable dropdowns: await
page.getByRole('combobox').fill('Europe'); await page.keyboard.press('Enter');
```

4 Asserting Selected State

```
// Assert selected value: await
expect(page.locator('#country')).toHaveValue('US');
// Assert all option labels available: const options
= await page.locator('#country
option').allInnerTexts();
expect(options).toEqual(['Select', 'US', 'Canada',
'UK']);
```

5 Auto-Scroll & Change Event Dispatch

Playwright automatically scrolls the selected option into view and dispatches both input and change DOM events so downstream reactive listeners update reliably.

6 Handling Bootstrap / AntD Hidden Selects

When native `<select>` is hidden with `display: none` by CSS frameworks, calling `selectOption()` with `{ force: true }` sets the underlying value without UI rendering errors!

"One unified method `selectOption()` covers label, value, index, and multi-selection!"

10. Mouse & Keyboard Actions

1 Mouse Actions & Hover

- **Hover:** `await page.getByText('Hover Menu').hover();`
- **Right Click:** `await page.click('#item', { button: 'right' });`
- **Double Click:** `await page.dblclick('#row');`

```
// Coordinate-based mouse control: await
page.mouse.move(100, 200); await page.mouse.down();
await page.mouse.move(300, 200, { steps: 10 }); //
Smooth glide await page.mouse.up();
```

2 Drag and Drop Strategies

```
// Approach A: High-level one-liner await
page.locator('#source-item')
  .dragTo(page.locator('#target-dropzone')); //
Approach B: Manual step-by-step for complex HTML5
canvas: const source = page.locator('#card-1');
const target = page.locator('#column-done'); await
source.hover(); await page.mouse.down(); await
target.hover(); await page.mouse.up();
```

3 Keyboard Press & Shortcuts

```
// Single Key Press: await
page.keyboard.press('Enter'); await
page.keyboard.press('ArrowDown'); await
page.keyboard.press('Tab'); // Modifier
Combinations: await
page.keyboard.press('Control+A'); await
page.keyboard.press('Control+C'); await
page.keyboard.press('Control+V'); // OS agnostic
modifier (Meta on Mac, Control on Windows): const
modifier = process.platform === 'darwin' ? 'Meta' :
'Control'; await
page.keyboard.press(`${modifier}+Shift+P`);
```

4 fill() vs pressSequentially()

Method	Mechanism	Ideal Scenario
<code>fill('txt')</code>	Clears input, sets value via DOM event instantly	High-speed form filling
<code>pressSequentially()</code>	Dispatches keyDown, keyPress, keyUp per char	Search auto-suggest, phone mask formatters

5 Scrolling & Mouse Wheel Mechanics

```
// Scroll vertical/horizontal via mouse wheel: await page.mouse.wheel(0, 600); // deltaX=0, deltaY=600px
// Auto-scroll element into viewport if hidden: await page.locator('#legal-disclaimer').scrollIntoViewIfNeeded();
```

6 Key Interview Tip: Handling Sticky Floating Headers During Clicks

If a sticky navbar intercepts clicks during scroll, use `position: { x: 5, y: 5 }` or scroll into view with margin offset before clicking to guarantee the element is not covered!

"Realistic OS-level mouse and keyboard dispatch with full step interpolation!"

11. Waits, Timers & Timeout Hierarchy

1 The Senior SDET Rule: BANISH HARD SLEEPS!

Never use `await page.waitForTimeout(5000)` in production automation code! It wastes hours of CI build time and fails randomly on loaded environments. Rely strictly on **Web-First Assertions** and explicit state predicates.

2 Explicit Wait Strategies

```
// 1. Wait for element DOM state: await
page.locator('.spinner').waitFor({ state: 'hidden'
}); await page.locator('#btn').waitFor({ state:
'visible', timeout: 8000 }); // 2. Wait for URL
match: await page.waitForURL('**/dashboard'); // 3.
Wait for network condition: await
page.waitForLoadState('networkidle'); // No traffic
for 500ms // 4. Wait for custom JS condition in
browser: await page.waitForFunction(() =>
window.document.readyState === 'complete');
```

3 The Complete Timeout Hierarchy

Test Timeout (Default: 30,000 ms)

Capped maximum for entire `test('name', ...)` block.

↓ governs

Action Timeout (Default: 0 ms / falls back to test timeout)

Per individual `.click()`, `.fill()`. Set via `actionTimeout`.

↓ alongside

Expect Assertion Timeout (Default: 5,000 ms)

Per assertion `await expect(...).toBeVisible()`.

4 Overriding Timeouts Dynamically

```
// Extend timeout for slow heavy task:
test('export 50k row CSV', async ({ page }) => {
test.setTimeout(120 * 1000); // 2 mins await
page.click('#export'); }); // Single assertion
override: await expect(page.getByText('Completed'))
.toBeVisible({ timeout: 15000 });
```

5 Polling with `toPass()` Block

```
// Retries entire block until success: await
expect(async () => { const resp = await
page.request.get('/status'); expect(await
resp.json()).toEqual({ status: 'PROCESSED' });
}).toPass({ timeout: 15000, intervals: [1000, 2000]
});
```

6 Interview Question: How does Auto-waiting differentiate Playwright from Cypress?

Cypress only auto-waits for element existence in DOM. Playwright performs full **Actionability verification**: ensuring the element is visible, non-moving, enabled, and actually receiving pointer events without intercepting overlays!

"Playwright auto-waits so you almost never need manual wait statements!"

12. Dynamic Elements & Synchronization

1 Web-First Assertions (Auto-Retrying)

Assertions re-poll DOM continuously until condition matches or timeout occurs:

```
// Retries until visible: await
expect(page.locator('.toast')).toBeVisible(); //
Retries until text equals expected: await
expect(page.locator('#balance')).toHaveText('$5,000.00'); // Inverse auto-retry (waits for spinner to detach): await
expect(page.locator('.spinner')).not.toBeVisible();
await expect(page.locator('#submit')).toBeEnabled();
```

2 Handling AJAX & Async Hydration

In React/Next.js apps, elements appear before click handlers attach. Synchronize with network:

```
// Wait for specific API response on click:
await Promise.all([ page.waitForResponse(res =>
res.url().includes('/api/items') && res.status() === 200 ), page.getByRole('button', { name: 'Load Data' }).click() ]);
```

3 Handling Dynamic Changing IDs (e.g. id="user_729482")

Dynamic Scenario	Fragile Anti-pattern	Playwright Recommended Solution
Random UUIDs	<code>page.locator('#input_f7d2')</code>	<code>page.getByLabel('First Name')</code>
Prefixed Dynamic IDs	<code>//input[@id='usr_123']</code>	<code>page.locator('input[id^="usr_"]')</code>
Shifting Dynamic Rows	<code>//tr[6]/td[3]</code>	<code>page.getByRole('row').filter({ hasText: 'Item 4' })</code>

4 Why Stale Element Exceptions Never Exist

In Selenium, saving an element reference before DOM re-render causes `StaleElementReferenceException`. In Playwright, Locators are **lazy queries** re-executed right at action invocation, automatically querying the fresh DOM tree!

5 Polling Dynamic API Status

```
// Poll backend status until ready: await
expect.poll(async () => { const res = await
page.request.get('/api/job/99'); return (await
res.json()).status; }, { message: 'Job status never reached COMPLETED', timeout: 10000,
}).toBe('COMPLETED');
```

6 Key Synchronizing Strategy: `waitForRequest` vs `waitForResponse`

Use `page.waitForRequest()` to verify client payload dispatch (e.g. analytics events). Use `page.waitForResponse()` to wait for backend computation before checking the updated UI!

"Zero Stale Element Reference Exceptions. Pure resilience by design!"

13. Screenshots, Videos & Visual Testing

1 Screenshot Capabilities

```
// 1. Visible Viewport: await
page.screenshot({ path: 'screenshots/view.png' });
// 2. Full Scrollable Page: await page.screenshot({
path: 'screenshots/full.png', fullPage: true }); //
3. Single Specific Element: await
page.locator('#chart').screenshot({ path:
'screenshots/chart.png' }); // 4. Return as binary
buffer (for AWS S3): const buffer = await
page.screenshot();
```

2 Visual Regression Comparisons

Pixel-by-pixel gold baseline comparison built right in:

```
// Compare against baseline gold standard:
await expect(page).toHaveScreenshot('dashboard.png',
{ maxDiffPixelRatio: 0.05, mask:
[page.locator('.live-ticker')] // ignore dynamic });
// Update snapshot baselines: // npx playwright test
--update-snapshots
```

3 Video Recording Configuration

```
// Configured in playwright.config.ts: use:
{ video: 'retain-on-failure', // 'on' | 'off' | 'on-
first-retry' } // Or in manual browser context:
const context = await browser.newContext({
recordVideo: { dir: 'videos/', size: { width: 1280,
height: 720 } } }); await context.close(); // Video
saves on close!
```

4 Playwright Trace Viewer (Best Debugger)

- Records live DOM snapshots, network payloads & console logs.
- Set trace: 'on-first-retry' in config.
- Open trace locally: `npx playwright show-trace trace.zip`
- Time-travel through before/after snapshots of every locator!

5 Masking Sensitive PII Elements

For HIPAA/GDPR compliance, mask passwords, credit cards, and timestamps during visual assertions:

```
await expect(page).toHaveScreenshot({ mask: [page.locator('#ssn-input'), page.locator('.avatar-img')],
maskColor: '#ff00ff' });
```

6 Trace Viewer vs Video: Interview Answer

Videos are passive recordings with no DOM inspection. **Trace Viewer** captures interactive serialized DOM snapshots, allowing you to open Chrome DevTools inside any past step to inspect elements, styles, and network calls!

"Trace Viewer is 100x better than video: inspect live DOM at every step!"

14. Cookies, Storage & Authentication Reuse

1 Storage State (Save & Reuse Login Session) — Core SDET Design

Never log in via UI inside each test! Log in once in setup, save cookies/localStorage to JSON, and inject into all tests:

```
// Step 1: auth.setup.ts (Runs once before test suite) import { test as setup } from '@playwright/test';
setup('authenticate', async ({ page, context }) => { await page.goto('/login'); await
page.getByLabel('User').fill('admin@corp.com'); await page.getByLabel('Pass').fill('Secret123'); await
page.getByRole('button', { name: 'Log in' }).click(); await page.waitForURL('/dashboard'); // Save session
state to JSON file: await context.storageState({ path: 'playwright/.auth/user.json' }); }); // Step 2: In
playwright.config.ts: use: { storageState: 'playwright/.auth/user.json' } // All tests start PRE-AUTHENTICATED!
```

2 Managing Browser Cookies

```
// Retrieve all cookies: const cookies =
await context.cookies(); // Inject custom cookie:
await context.addCookies([{ name: 'auth_jwt', value:
'xyz987token', domain: 'app.example.com', path: '/',
httpOnly: true, secure: true }]); // Clear all
cookies: await context.clearCookies();
```

3 Local & Session Storage Access

```
// Read localStorage: const token = await
page.evaluate(() => localStorage.getItem('token') );
// Inject localStorage: await page.evaluate(() => {
localStorage.setItem('has_accepted_terms', 'true');
}); // Clear storage: await page.evaluate(() =>
sessionStorage.clear());
```

4 Multi-Role Concurrent Testing

```
// Admin and Customer in 1 test! const
adminContext = await browser.newContext({
storageState: 'admin.json' }); const userContext =
await browser.newContext({ storageState: 'user.json'
}); const adminPage = await adminContext.newPage();
const userPage = await userContext.newPage(); //
Test live real-time chat between both!
```

5 Bypassing UI Login Completely via API

```
// Faster: Auth via API request fixture:
const res = await request.post('/api/login', { data:
creds }); // Seed cookies into context directly!
```

6 Interview Question: How to handle 2FA (Two-Factor Authentication) in CI?

Generate TOTP tokens programmatically using `otplib` and the shared TOTP secret key, or seed the authenticated `storageState.json` before running regression tests!

"Log in once via setup, reuse storageState for 10x faster execution!"

15. Network Interception & Browser Options

1 Route & Mock Network Requests

- Intercept outgoing HTTP requests and return mock JSON fixtures instantly.
- Eliminates dependency on slow or unstable 3rd-party backend staging services.

```
// Intercept route and stub response: await page.route('**/api/v1/user/profile', async route => { await route.fulfill({ status: 200, contentType: 'application/json', body: JSON.stringify({ name: 'Mocked John', tier: 'VIP_PLATINUM' }) }); }); await page.goto('/profile'); // Shows "VIP_PLATINUM" immediately!
```

2 Abort & Block Heavy Assets

```
// Speed up CI runs by blocking images/fonts: await page.route('**/*.{png,jpg,jpeg,svg,gif}', route => { route.abort(); // or route.abort('blockedbyclient') }); // Block analytics trackers: await page.route('**/google-analytics.com/**', route => route.abort());
```

3 Modifying Live Requests/Headers

```
// Append custom authentication header: await page.route('**/api/**', async route => { const headers = { ...route.request().headers(), 'x-qabypass': 'true' }; await route.continue({ headers }); });
```

4 Simulating Offline & Throttling

```
// Context offline mode: await context.setOffline(true); await expect(page.getByText('No Internet')).toBeVisible(); // Chrome DevTools Protocol 3G throttling: const cdp = await context.newCDPSession(page); await cdp.send('Network.emulateNetworkConditions', { offline: false, downloadThroughput: (500 * 1024) / 8, // 500 kbps uploadThroughput: (500 * 1024) / 8, latency: 400 });
```

5 Capturing Console & Unhandled Errors

```
// Fail test if JS console error occurs: page.on('console', msg => { if (msg.type() === 'error') console.error(`Client JS error: ${msg.text()}`); }); // Catch unhandled page exceptions: page.on('pageerror', error => { console.log(`Uncaught exception: ${error.message}`); });
```

6 Mocking HAR (HTTP Archive) Playback

Record complete network traffic once into HAR file and replay it in CI for offline reproducibility:

```
await page.routeFromHAR('fixtures/network.har', { update: false });
```

"Mock APIs and block tracking scripts for ultra-reliable, lightning fast runs!"

16. Handling Dynamic Web Tables

1 Web Table Navigation Philosophy

- Tables are dynamic: rows sort, paginate, and shift indices. Never hardcode row numbers like `//tr[4]!`
- Locate table rows dynamically using `getByRole('row').filter({ hasText: 'Unique Key' })`.

2 Locate Row by Cell Text & Click Action

```
// 1. Target table container: const table = page.getByRole('table'); // 2. Filter specific row
containing unique text (e.g., 'Alice Smith'): const targetRow = table.getByRole('row').filter({ hasText: 'Alice
Smith' }); // 3. Perform action scoped inside THAT row: await targetRow.getByRole('button', { name: 'Edit'
}).click(); // 4. Assert value inside a specific column cell of that row: const roleCell =
targetRow.getByRole('cell').nth(2); await expect(roleCell).toHaveText('QA Lead');
```

3 Extracting Complete Table to 2D Array

```
const rows = page.locator('table#empTable tbody tr'); const count = await rows.count(); const
tableData = []; for (let i = 0; i < count; i++) {
const cells = rows.nth(i).locator('td'); const
values = await cells.allInnerTexts();
tableData.push(values); } console.log(tableData);
```

4 Handling Paginated Tables Loop

```
let found = false; while (!found) { const
item = page.getByRole('cell', { name: 'Target SKU'
}); if (await item.isVisible()) { await
item.click(); found = true; break; } const nextBtn =
page.getByRole('button', { name: 'Next' }); if
(await nextBtn.isDisabled()) break; await
nextBtn.click(); }
```

5 Table Column Sorting Validation

```
// Click header to sort A-Z: await
page.getByRole('columnheader', { name: 'Price'
}).click(); const prices = await
page.locator('td.col-price').allInnerTexts(); const
sorted = [...prices].sort((a, b) => parseFloat(a) -
parseFloat(b)); expect(prices).toEqual(sorted);
```

6 Handling Infinite Scroll Tables

```
while (await page.getByText('Target
Item').isHidden()) { await page.mouse.wheel(0,
1000); await page.waitForTimeout(300); }
```

"Always filter row by unique record key before locating internal action buttons!"

17. Calendars, Date Pickers & Complex Elements

1 Approach A: Direct Input Fill (Fastest)

If input is not readonly, fill formatted date string directly:

```
const dateInput = page.getByLabel('Departure Date'); await dateInput.fill('2026-10-25'); await dateInput.press('Enter');
```

Always check this first — saves 10+ unnecessary UI clicks!

2 Approach B: Interactive Date Picker Loop

```
// Open calendar widget: await
page.locator('#calendar-picker').click(); // Loop
next month until target visible: while (!await
page.getByText('October 2026').isVisible()) { await
page.getByRole('button', { name: 'Next Month'
}).click(); } // Select exact day: await
page.getByRole('button', { name: '25', exact: true
}).click();
```

3 Handling File Uploads (Native & Hidden)

Standard input[type="file"]:

Playwright uploads directly even if element is hidden/invisible!

```
await page.locator('input[type="file"]')
.setInputFiles('data/resume.pdf'); // Multiple files
upload: await page.locator('input[type="file"]')
.setInputFiles(['doc1.pdf', 'doc2.pdf']);
```

Custom Drag-and-Drop or Button:

Wait for filechooser event!

```
const [fileChooser] = await Promise.all([
page.waitForEvent('filechooser'),
page.getByText('Upload Receipt').click() ]); await
fileChooser.setFiles('receipt.png');
```

4 Handling File Downloads

```
const [download] = await Promise.all([
page.waitForEvent('download'),
page.getByRole('button', { name: 'Export CSV'
}).click() ]); // Save downloaded file to target
directory: const filePath = await download.path();
await download.saveAs('./downloads/report.csv');
```

5 Sliders & Range Inputs

```
const slider =
page.locator('input[type="range"]'); // Approach 1:
Arrow keys await slider.focus(); await
page.keyboard.press('ArrowRight'); // Approach 2:
Fill value directly await slider.fill('75');
```

6 Canvas & Signature Pads

Interact with HTML5 canvas by moving the mouse with `down()` and `up()` events:

```
const box = await page.locator('#signature-pad').boundingBox(); await page.mouse.move(box.x + 20, box.y
+ 20); await page.mouse.down(); await page.mouse.move(box.x + 100, box.y + 80); await page.mouse.up();
```

"setInputFiles handles hidden file inputs without needing third-party tools!"

18. Four Signature Superpower Features

1 Playwright Codegen (Test Generator)

- Interactive browser records your manual actions directly into resilient test code.
- Launch: `npx playwright codegen https://app.dev`
- Emulate mobile device: `--device="iPhone 14 Pro"`
- Preserve auth session: `--save-storage=auth.json`
- Export code to TypeScript, JavaScript, Python, Java, or C#.

2 Playwright UI Mode (Watch & Time-Travel)

- Launch: `npx playwright test --ui`
- Time-travel through every locator action with live before/after DOM diffs.
- Watch mode automatically re-runs specs on code save.
- Built-in visual DOM locator picker and console/network panel.

3 Unified API Testing (request fixture)

Zero need for external Axios/Supertest. Shares cookies with UI context:

```
test('API creation check', async ({ request }) => {
  const response = await request.post('/api/users', {
    data: { name: 'Alice', role: 'Tester' }
  });
  expect(response.status()).toBe(201);
  const body = await response.json();
  expect(body.id).toBeDefined();
});
```

4 Component Testing (CT Engine)

- Test React, Vue, Svelte components in real browser engines (not synthetic JSDOM).
- Fast Vite-powered test bundle.
- Import component directly: `const cmp = await mount(<Button label="Pay"/>)`.
- Validates real CSS rendering, fonts, and bounding boxes.

5 Comparison Matrix for Interviews

Feature	Core Problem Solved	SDET Productivity Impact
Codegen	Slow manual authoring	Generates ARIA-compliant locators in seconds
UI Mode	Headless blind debugging	Visual time-travel step inspection and instant locator picking
API Fixture	Slow UI-based test setup	Pre-seeds database records via REST in milliseconds
CT Engine	Unreliable synthetic unit DOM	Tests UI components against real browser rendering engines

"Codegen, UI Mode, API testing & CT make Playwright a complete QA platform!"

19. Page Object Model (POM) & BDD

1 Modern TypeScript POM Implementation

```
import { Page, Locator } from '@playwright/test'; export class LoginPage { readonly page: Page; readonly usernameInput: Locator; readonly passwordInput: Locator; readonly submitButton: Locator; constructor(page: Page) { this.page = page; this.usernameInput = page.getByLabel('Email address'); this.passwordInput = page.getByLabel('Password'); this.submitButton = page.getByRole('button', { name: 'Sign in' }); } async goto() { await this.page.goto('/login'); } async login(user: string, pass: string) { await this.usernameInput.fill(user); await this.passwordInput.fill(pass); await this.submitButton.click(); } }
```

2 Fixture-Based POM Injection

```
// Extend test with custom fixtures (No manual instantiation!): export const test = base.extend<{ loginPage: LoginPage }>({ loginPage: async ({ page }, use) => { const loginPage = new LoginPage(page); await use(loginPage); }, }); // Inside test file: test('valid login', async ({ loginPage }) => { await loginPage.goto(); await loginPage.login('qa@dev.com', 'Pass123'); });
```

3 BDD Integration with Cucumber

```
# features/login.feature Feature: User Login Scenario: Successful authorization Given I navigate to login page When I enter valid credentials Then I should see the dashboard
```

```
// step-definitions/login.steps.ts: Given('I navigate to login page', async function () { await this.page.goto('/login'); });
```

4 Golden POM Rules in Playwright

- **Store Locators, not ElementHandles:** In constructor, initialize `this.btn = page.locator()` (lazy).
- **No Assertions in Page Classes:** Keep assertions in test specs so stack traces point to test cases!

5 Playwright Test Runner vs Cucumber

While Cucumber offers Gherkin for non-technical stakeholders, the native Playwright test runner offers superior speed, built-in tracing, native fixture injection, and project dependencies.

6 Playwright-BDD Tool (Modern Hybrid)

Using `playwright-bdd` npm package compiles Gherkin feature files directly into native Playwright test specs, combining BDD syntax with 100% native Playwright runner capabilities!

"Wrap POMs inside test fixtures for clean, zero-boilerplate test specs!"

20. End-to-End Automation Framework Architecture

1 Enterprise Directory Structure

```
├── .github/workflows/e2e-ci.yml # GitHub Actions matrix execution pipeline ── config/ ──  
├── env.staging.json # Staging environment credentials & endpoints ── env.prod.json # Production smoke endpoints  
├── fixtures/ ── baseTest.ts # Extended fixtures (auth, POMs, API clients) ── pages/ # Page Object Models ──  
├── LoginPage.ts ── CheckoutPage.ts ── tests/ # E2E Test Specs ── auth.setup.ts # One-time login storage  
├── state generator ── checkout.spec.ts ── utils/ # DB connectors, Faker generators, Mailhog API ──  
├── playwright.config.ts # Global framework configuration & projects ── package.json
```

2 GitHub Actions CI Matrix Pipeline

```
name: Playwright Regression on: [push,  
pull_request] jobs: test: runs-on: ubuntu-latest  
steps: - uses: actions/checkout@v4 - uses:  
actions/setup-node@v4 with: { node-version: 18 } -  
run: npm ci - run: npx playwright install --with-  
deps - run: npx playwright test - uses:  
actions/upload-artifact@v4 if: always() with: name:  
playwright-report path: playwright-report/
```

3 Test Data Management

- **Dynamic Data:** Use `@faker-js/faker` for random unique emails, phones, and names.
- **API Seeding:** Seed precondition test records via API in `beforeAll` hooks rather than UI clicks.
- **Database Cleanup:** Clean up test data after execution via direct SQL queries or teardown API calls.

4 Multi-Project Dependencies

```
projects: [ { name: 'setup', testMatch:  
./.*\setup\*.ts/ }, { name: 'chromium', dependencies:  
['setup'], use: { ...devices['Desktop Chrome'],  
storageState: 'auth.json' } } ]
```

5 Sharding Tests Across CI Nodes

```
// Split 1000 tests across 4 CI machines:  
npx playwright test --shard=1/4 npx playwright test  
--shard=2/4 // Merges all reports at the end!
```

6 Monorepo & Multi-Environment Best Practice

Use `process.env.ENV` to toggle base URLs dynamically without changing code: `baseUrl: process.env.BASE_URL || 'https://staging.app.com'`.

"Modular, fixture-driven architecture that scales seamlessly to thousands of tests!"

21. SDET Best Practices & Anti-Patterns

1 DO'S (Proven Industry Best Practices)

- **Use user-facing locators:** `getByRole`, `getByLabel`.
- **Keep tests isolated:** Every test runs with its own fresh `BrowserContext`.
- **Use web-first assertions:** `await expect(locator).toHaveText()`.
- **Reuse authentication state:** Save session via `storageState`.
- **Enable Tracing on failure:** Invaluable for CI post-mortems!
- **Use soft assertions:** `expect.soft()` for non-blocking UI checks.

2 DON'TS (Dangerous Anti-Patterns)

- **Never use `waitForTimeout()`:** Introduces slow, flaky tests.
- **Avoid brittle CSS/XPaths:** e.g. `//div[3]/span/input`.
- **Do not depend on previous test:** Test order execution must never matter.
- **Don't test 3rd party services:** Mock third-party APIs (Stripe, Twilio).
- **Don't query static elements with `page.$`:** Bypasses auto-wait.

3 Soft Assertions Pattern (`expect.soft`)

Allows test execution to continue even if an assertion fails, compiling all errors at test completion:

```
// Test continues even if header or badge is wrong: await
expect.soft(page.getByRole('heading')).toHaveText('Account Overview'); await expect.soft(page.locator('.user-
status')).toHaveText('Active VIP'); // Critical assertion that stops execution if failed: await
expect(page.getByRole('button', { name: 'Transfer Funds' })).toBeVisible();
```

4 Test Tagging & Filtering

```
test('smoke checkout @smoke', async ({ page
}) => { ... }); test('nightly billing @regression',
async () => { ... }); // CLI Filtering: // npx
playwright test --grep "@smoke" // Invert: npx
playwright test --grep-invert "@slow"
```

5 Flaky Test Quarantine

- Use `test.fixme()` to skip known broken bugs until patched.
- Use `test.describe.configure({ retries: 2 })` to isolate intermittent specs during investigation.

6 Parameterized / Data-Driven Testing

```
const users = [ { role: 'admin', url: '/admin' }, { role: 'editor', url: '/editor' } ]; for (const user
of users) { test(`verify dashboard for ${user.role}`, async ({ page }) => { await page.goto(user.url); }); }
```

"Reliability > Speed > Test Count. Zero flakiness is the standard!"

22. Rapid-Fire Interview Revision Cheatsheet

Q1 How does Playwright talk to browsers?

Ans: Via a single persistent bi-directional **WebSocket** connection communicating over native debugging protocols (CDP for Chromium, Juggler for Firefox, WebKit protocol) directly with the browser engine without HTTP driver bottlenecks.

Q2 Browser vs Context vs Page?

Ans:

- **Browser:** Heavy binary OS process (shared per worker).
- **Context:** Lightweight isolated incognito profile (~10ms to spawn).
- **Page:** Single web page tab inside a context.

Q3 What are Web-First Assertions?

Ans: Assertions like `await expect(locator).toBeVisible()` that continuously re-poll the DOM until the condition matches or timeout expires, automatically eliminating animation and network race conditions.

Q4 How does Playwright prevent flakiness?

Ans: 1) Built-in **Actionability checks** (visibility, stability, enablement), 2) Isolated **BrowserContext** eliminating cookie leakage, 3) Auto-retrying assertions, 4) Direct **WebSocket** protocol responsiveness.

Q5 Top Tricky Interview Scenario Flashcards

Q: How do you bypass Cloudflare bot detection or 2FA in automated CI tests?

Ans: Generate authentication cookies/tokens via API or local login once and inject via `storageState.json`, or utilize dedicated test environment bypass API credentials.

Q: A button is obscured by a floating promotional banner. How do you click it?

Ans: Remove the banner via `page.evaluate(() => document.querySelector('.banner').remove())` or use `locator.click({ force: true })`.

Q: How do you simulate a backend 500 error to test error boundary toast UI?

Ans: `await page.route('**/api/data', route => route.fulfill({ status: 500 }));`

Q6 How to debug CI-only test failures?

Ans: Download `trace.zip` and inspect with `npx playwright show-trace trace.zip`. Gives live DOM snapshots, network waterfalls, and console logs exactly as captured on CI!

Q7 How to handle memory leaks in large suites?

Ans: Close pages/contexts in `afterEach` hooks, avoid storing heavy DOM references in global arrays, and tune worker process counts (e.g. `workers: 2` in CI).

Q8 Difference between `page.waitForLoadState()` states?

State	Trigger Event	Recommended Usage
load	window.onload fired	Standard multi-page apps
domcontentloaded	HTML parsed, scripts executed	Fast navigation checks
networkidle	No network calls for 500ms	SPA post-render (use with care)

"Master the architecture, articulate the trade-offs, and crack your SDET interview!"